

Introduction to Algorithms

Levon Prazyan

Course Overview

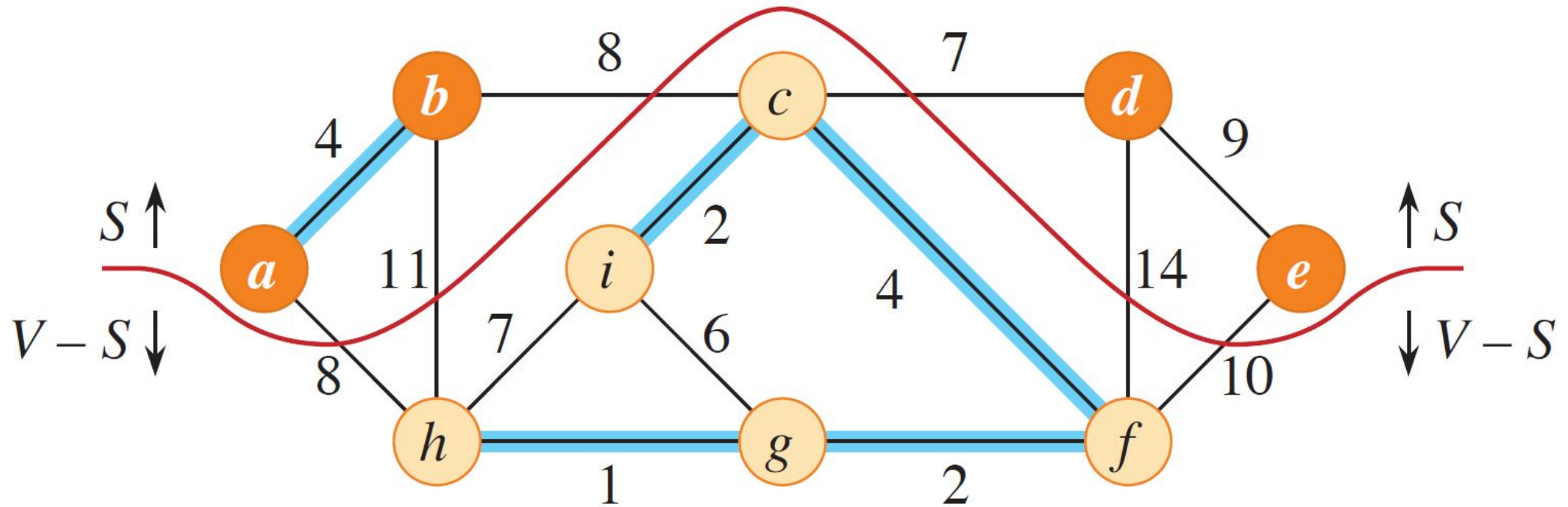
- Introduction
- Sorting and Searching
- Arrays and Lists
- Stacks and Queues, Double Ended Queue
- Heaps
- Trees
- Hash Tables
- Dynamic Programming
- Greedy Algorithms
- Graph Algorithms
- Network Flow
- NP-Completeness

Definitions

Given an undirected graph $G = (V, E)$

- A **cut** $(S, V - S)$ is a partition of V .
- An edge $(u, v) \in E$ **crosses** the cut $(S, V - S)$ if one endpoint belongs to S and the other belongs to $V - S$.
- A cut **respects** a set A of edges if no edges from set A crosses the cut.
- An edge is a **light edge** satisfying a given property if its weight is the minimum of any edge satisfying the property.

Example



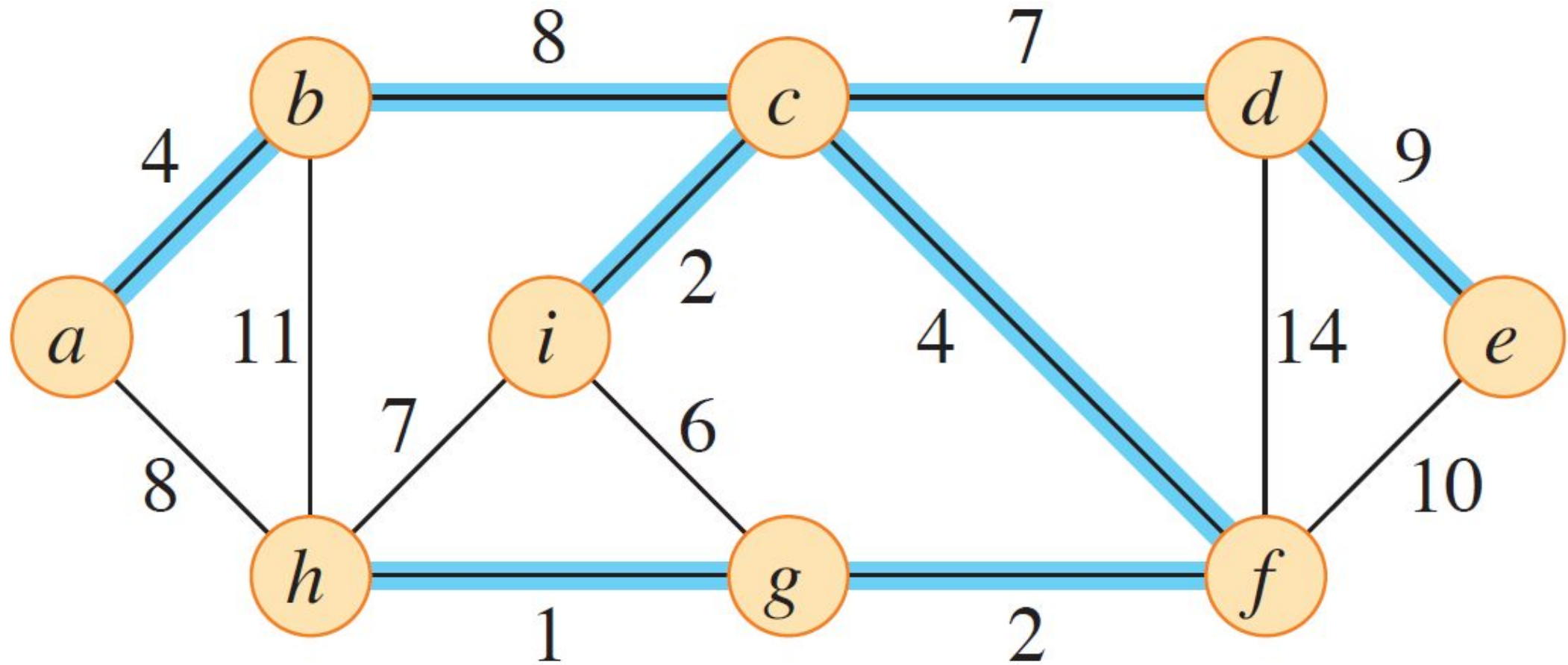
Minimum Spanning Tree (MST)

-

Given a connected and undirected graph $G = (V, E)$ and for any $(u, v) \in E$ a weight w specifies a cost to connect u to v . The goal is to find an acyclic subset $T \subseteq E$ that connects all vertices and whose total weight is **minimized**.

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

Minimum Spanning Tree Example



Growing an MST

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

Growing an MST: Proof

Prior to each iteration, A is a subset of some minimum spanning tree

- **Initialization:** After line 1, the set A trivially satisfies the loop invariant.
- **Maintenance:** The loop in lines 2, 3, 4 maintains the invariant by adding only safe edges.
- **Termination:** All edges added to A belong to a minimum spanning tree, and the loop must terminate by the time it has considered all edges. Therefore, the set A returned in line 5 must be a minimum spanning tree.

Growing an MST: Safe Edges Theorem

Theorem:

Given a connected and undirected graph $G = (V, E)$.

- w is a real-valued weight function defined on E .
- Let A be a subset of E that is included in some MST.
- Let $(S, V - S)$ be any cut of G that respects A .
- Let (u, v) be a light edge crossing $(S, V - S)$.

Then edge (u, v) is safe for A .

Growing an MST: Safe Edges Theorem: Proof

- Let T be a minimum spanning tree that includes A , and $(u, v) \notin T$.
- There is a simple path p from u to v , laying in T , and containing an edge crossing the cut $(S, V - S)$. Let (x, y) be such edge.
- Let's construct a tree $T' = (T - \{x, y\}) \cup \{u, v\}$
- As (u, v) is a light edge, $w(u, v) \leq w(x, y)$
- $w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$
- T is an MST, so T' is an MST as well

MST Algorithms

Joseph Kruskal



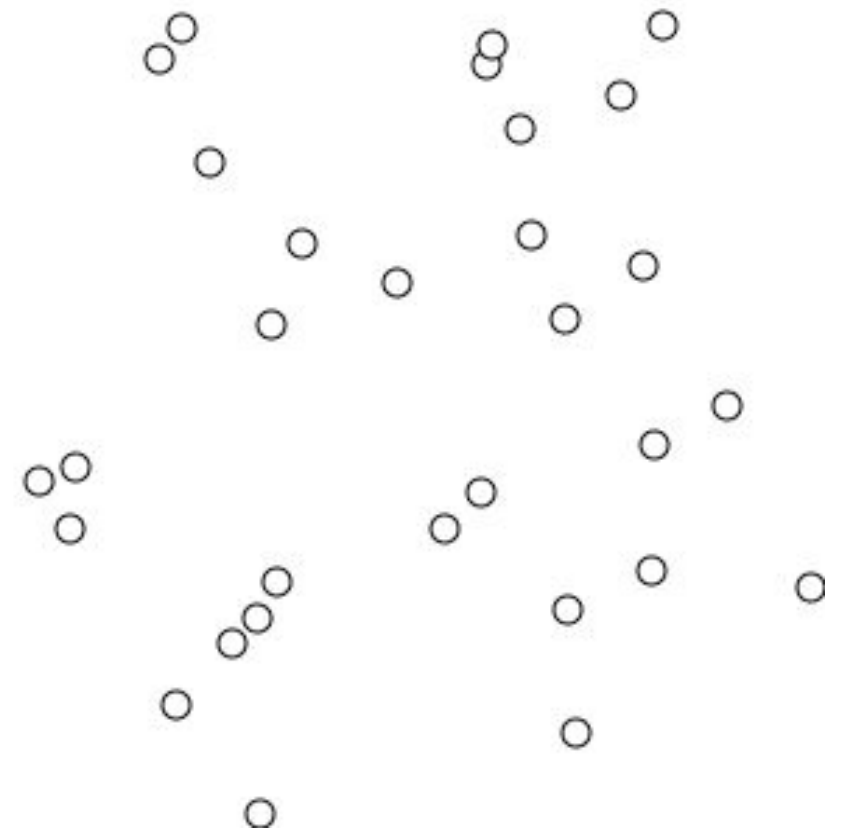
Vojtěch Jarník, Robert Prim



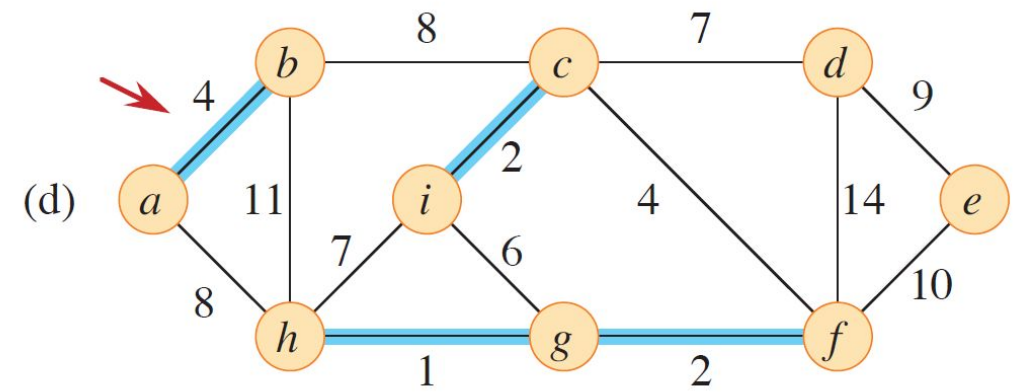
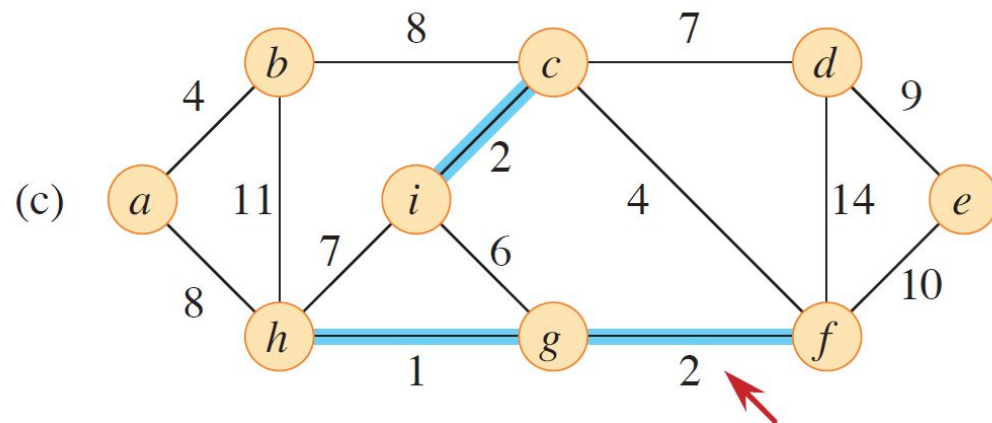
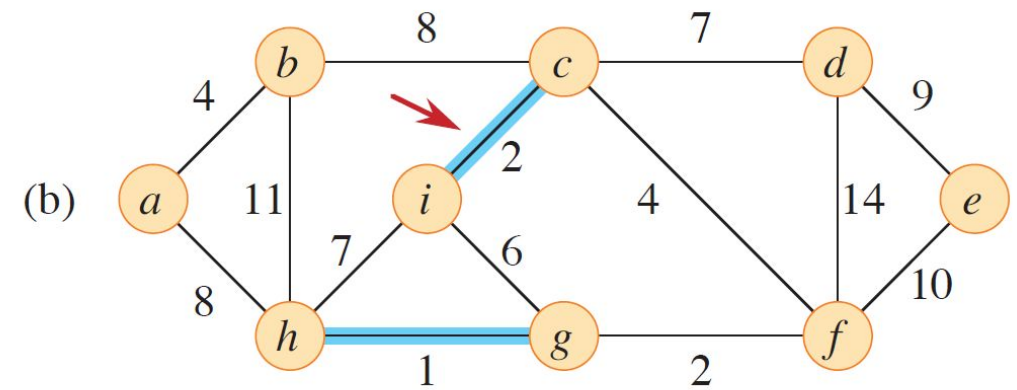
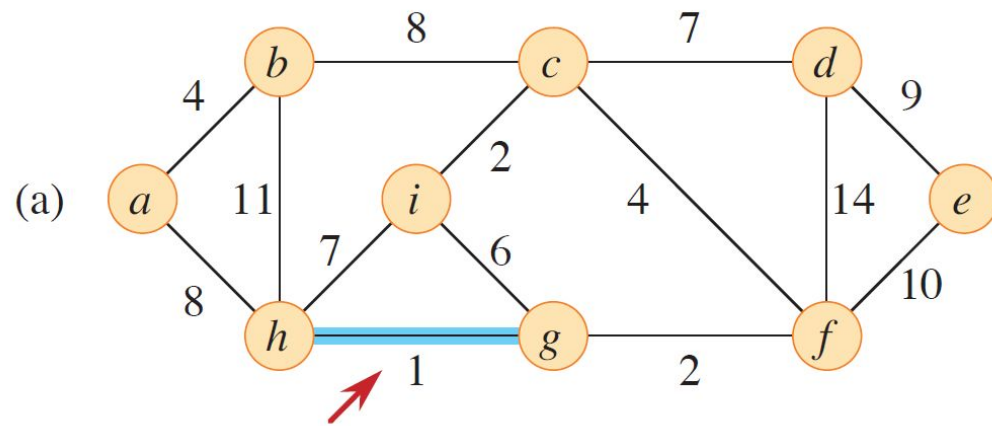
MST: Kruskal Algorithm

Defines a rule to determine a safe edge in *Generic – MST* as follows:

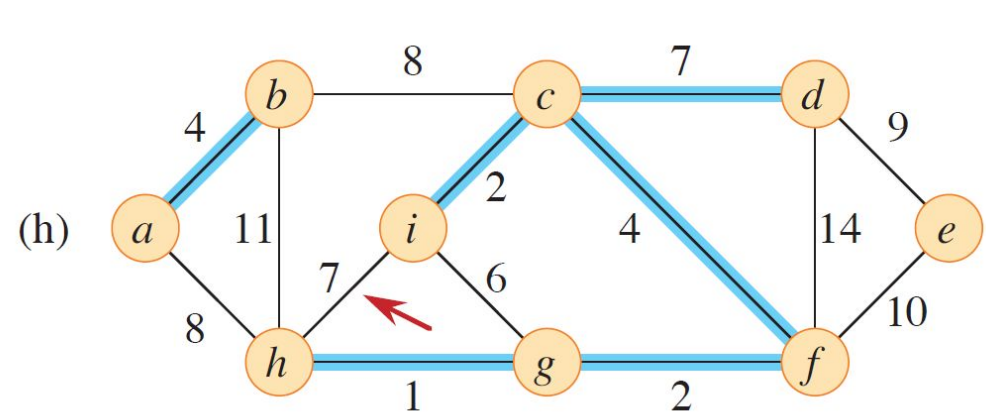
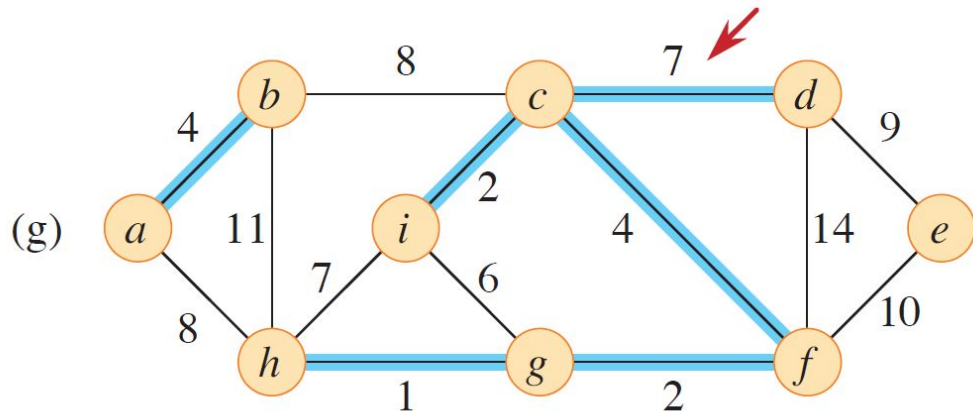
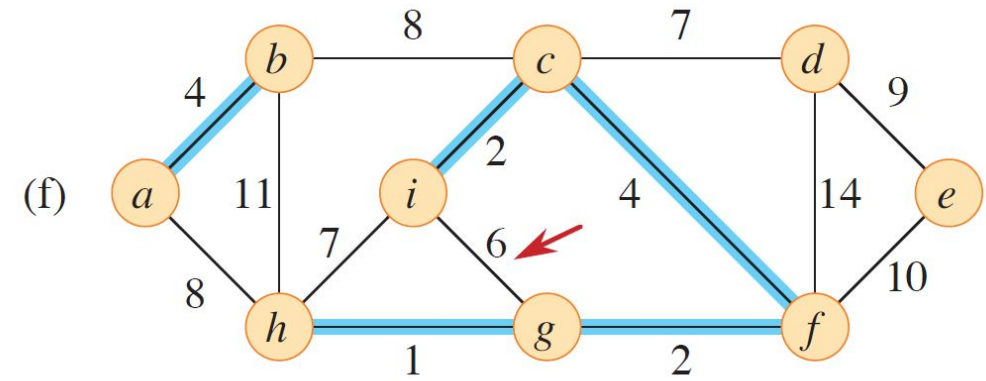
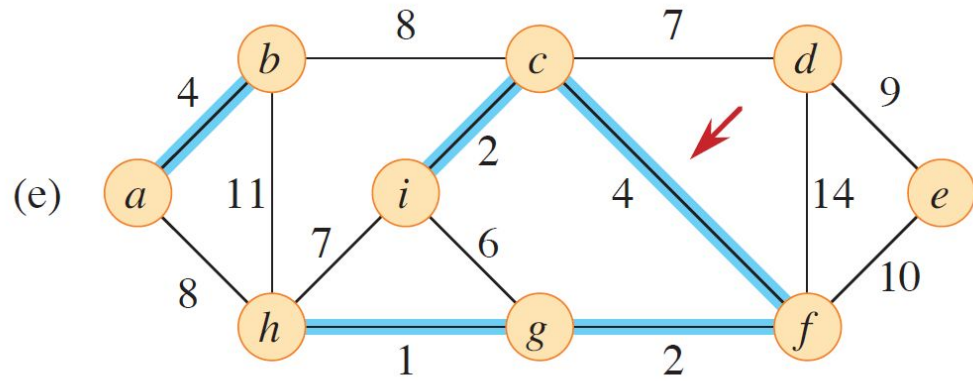
- the set A is a forest whose vertices are all those of the given graph,
- safe edge added to A is always a lowest-weight edge in the graph that connects two distinct components



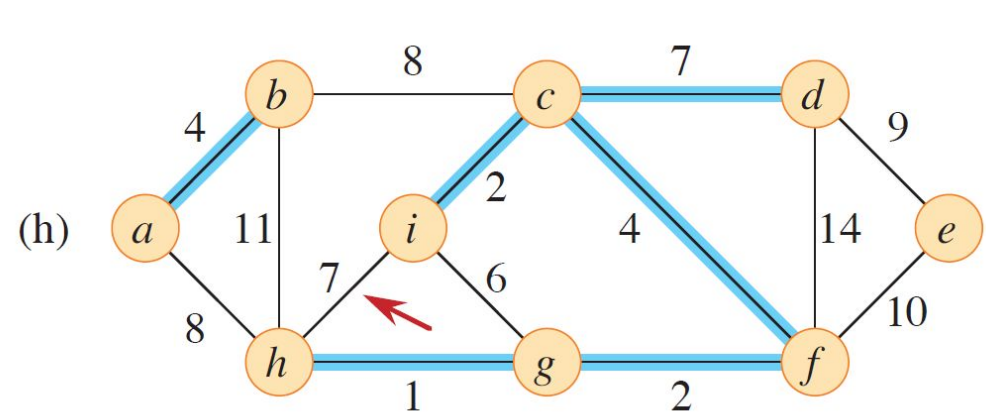
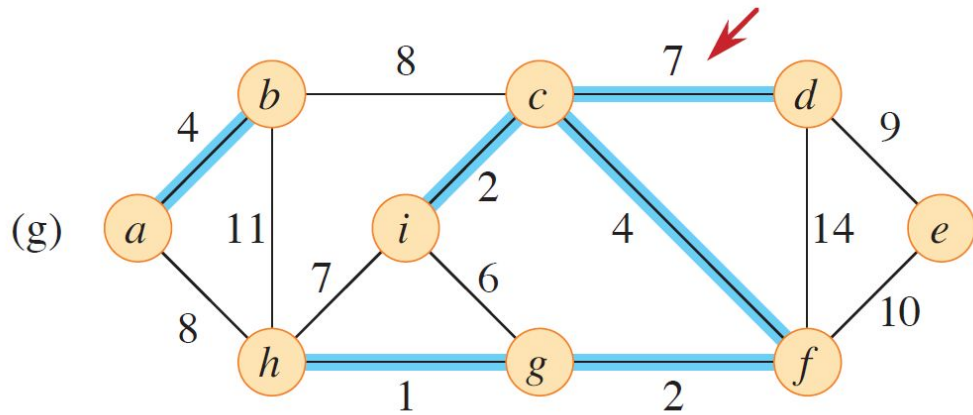
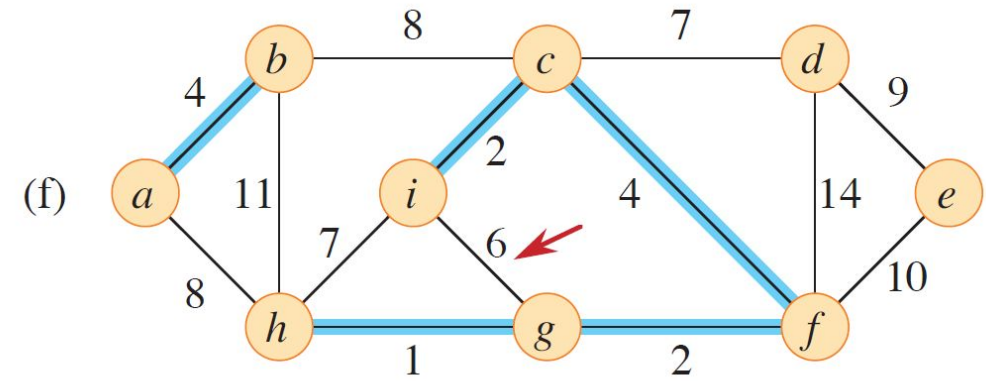
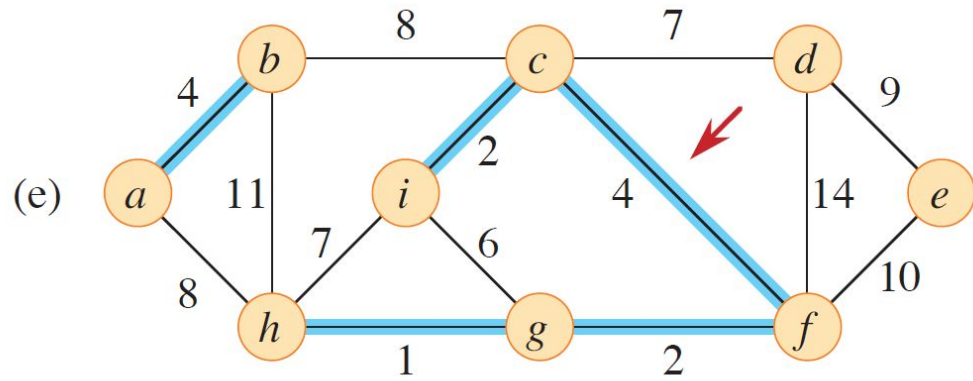
MST: Kruskal Algorithm: Example (1)



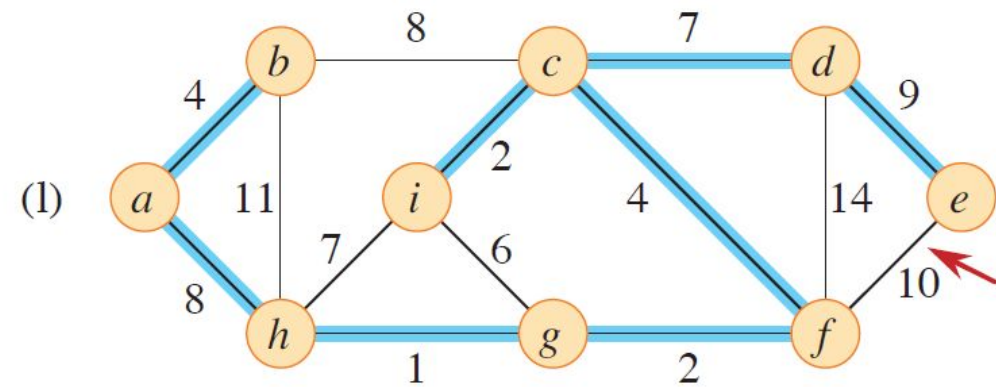
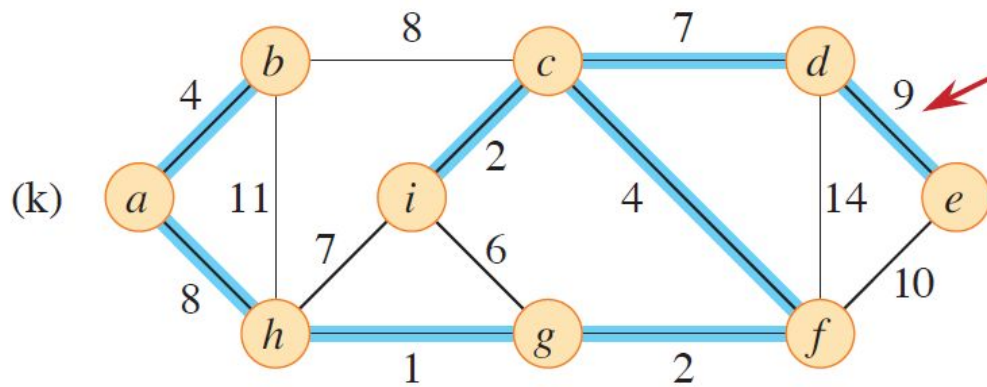
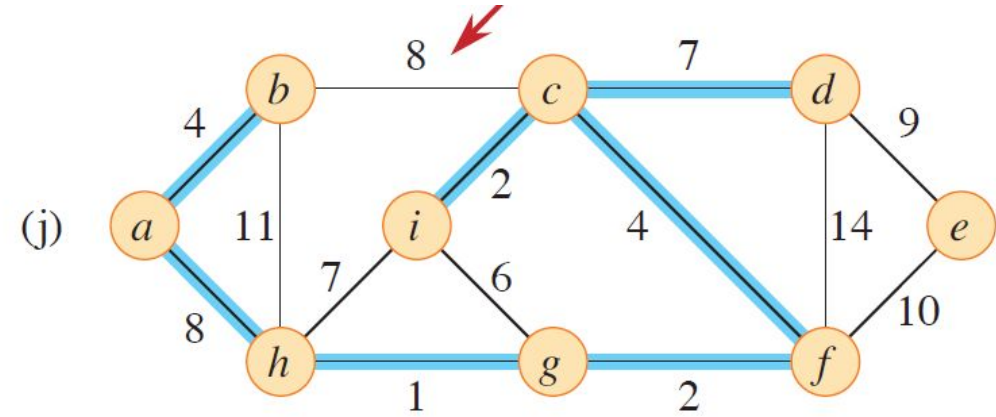
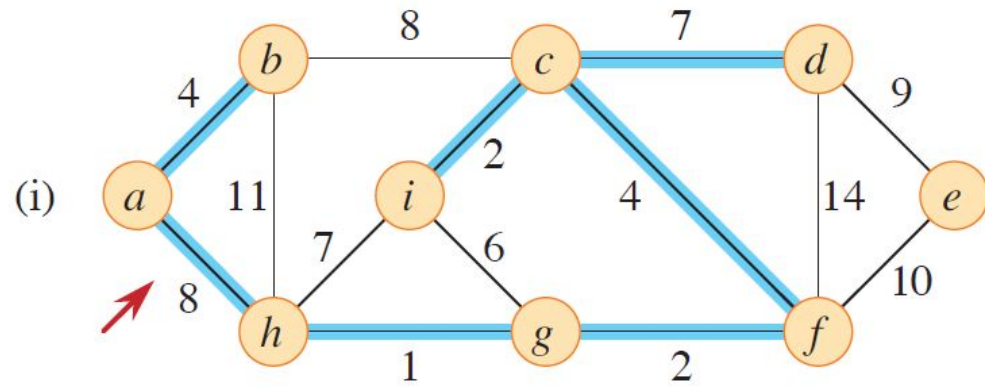
MST: Kruskal Algorithm: Example (2)



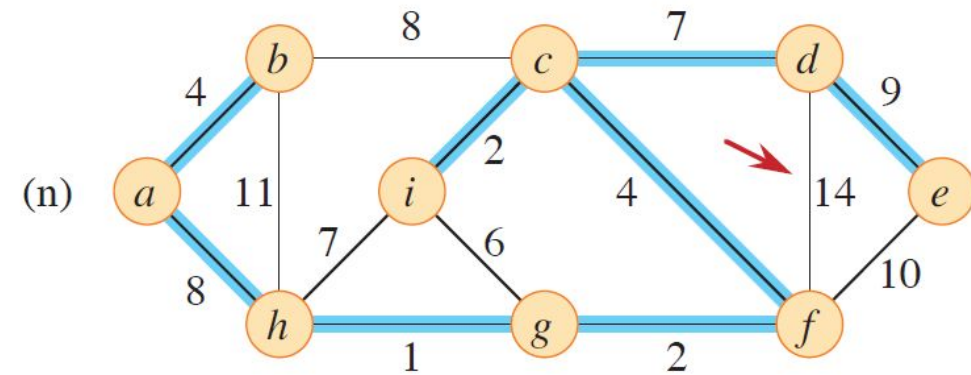
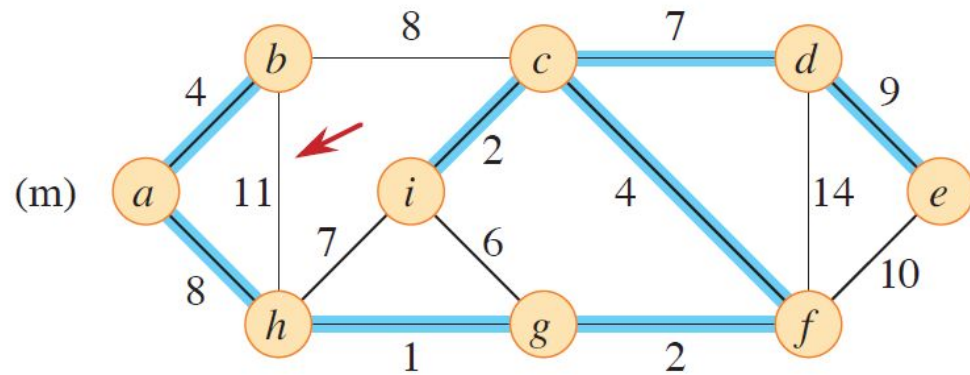
MST: Kruskal Algorithm: Example (3)



MST: Kruskal Algorithm: Example (4)



MST: Kruskal Algorithm: Example (5)



MST: Kruskal Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  create a single list of the edges in  $G.E$ 
5  sort the list of edges into monotonically increasing order by weight  $w$ 
6  for each edge  $(u, v)$  taken from the sorted list in order
7      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
8           $A = A \cup \{(u, v)\}$ 
9          UNION( $u, v$ )
10 return  $A$ 
```

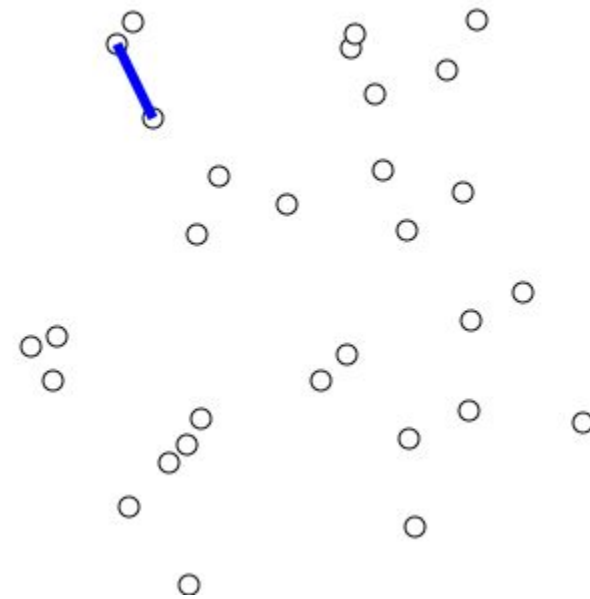
MST: Kruskal Algorithm: Analysis

```
MST-KRUSKAL( $G, w$ )
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  create a single list of the edges in  $G.E$ 
5  sort the list of edges into monotonically increasing order by weight  $w$ 
6  for each edge  $(u, v)$  taken from the sorted list in order
7      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
8           $A = A \cup \{(u, v)\}$ 
9          UNION( $u, v$ )
10 return  $A$ 
```

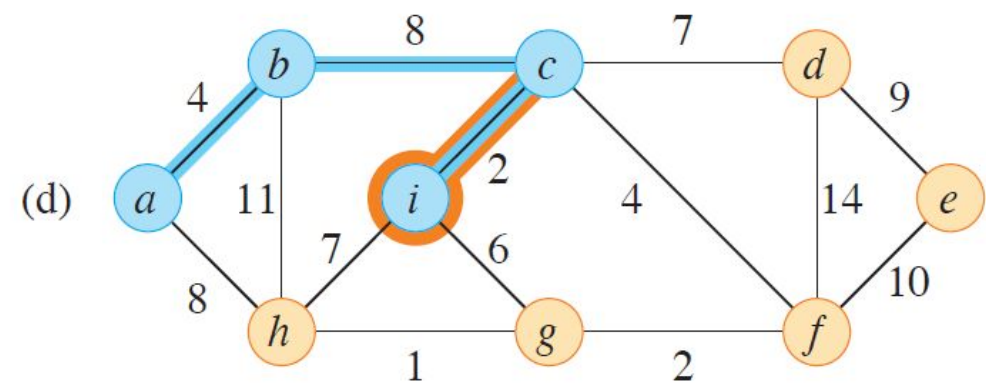
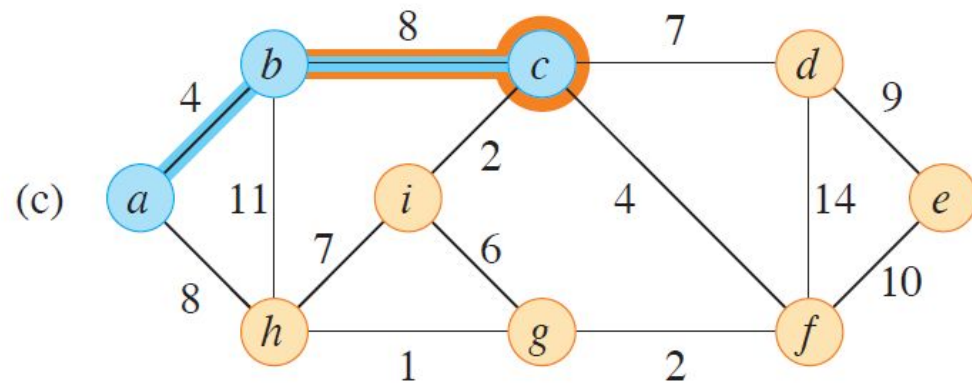
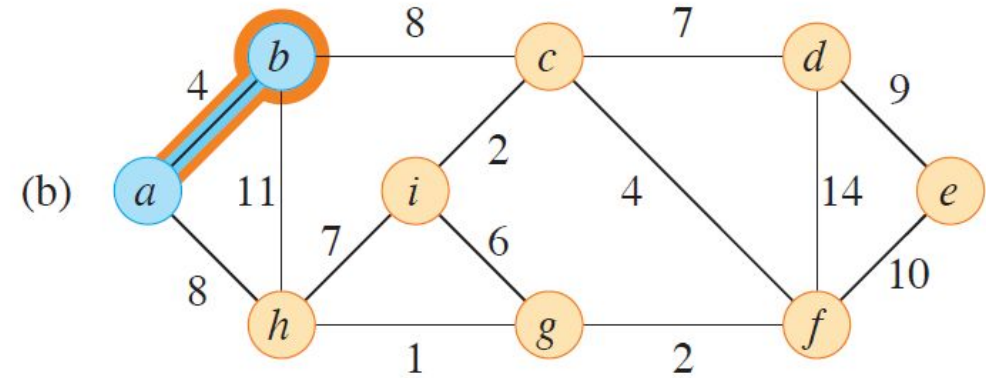
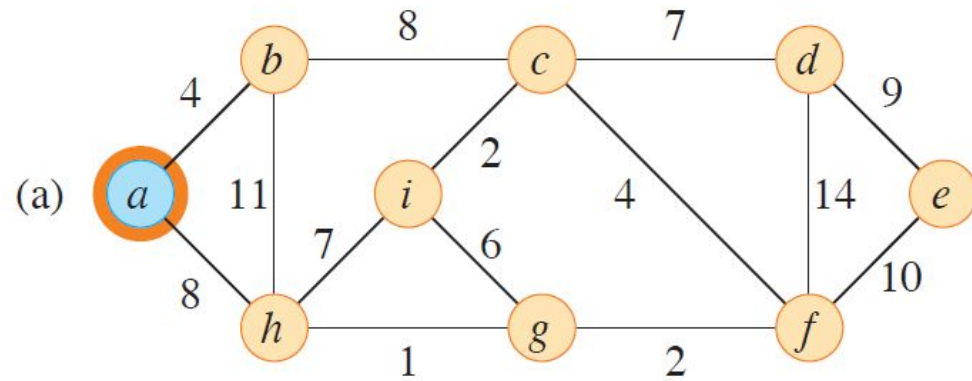
- Line 1 – $O(1)$ time.
- Line 4 – $O(V + E)$ time.
- Line 5 – $O(E \log E)$ time.
- Line 6-9 is called $|E|$ times.
- $|V|$ times called Make_Set and $|E|$ times Find_Set and Union take $O((V + E)\alpha(V))$ time.
- $|E| \geq |V| - 1$; $\alpha(V) = O(\log V)$
- Total time - **$O(E \log V)$**

MST: Prim Algorithm

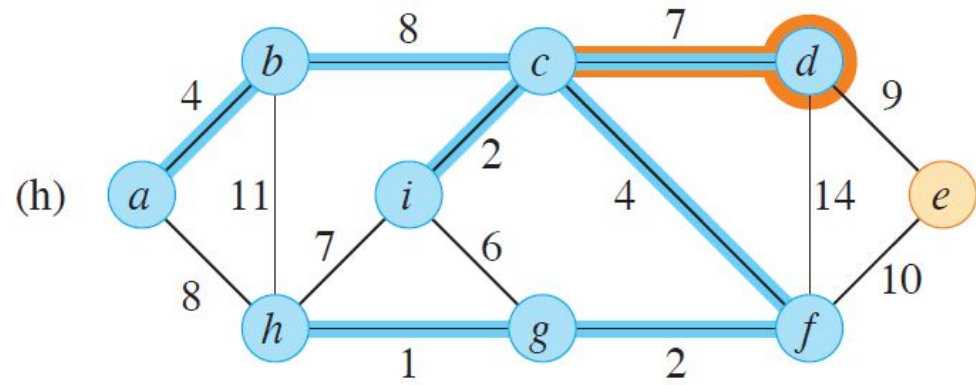
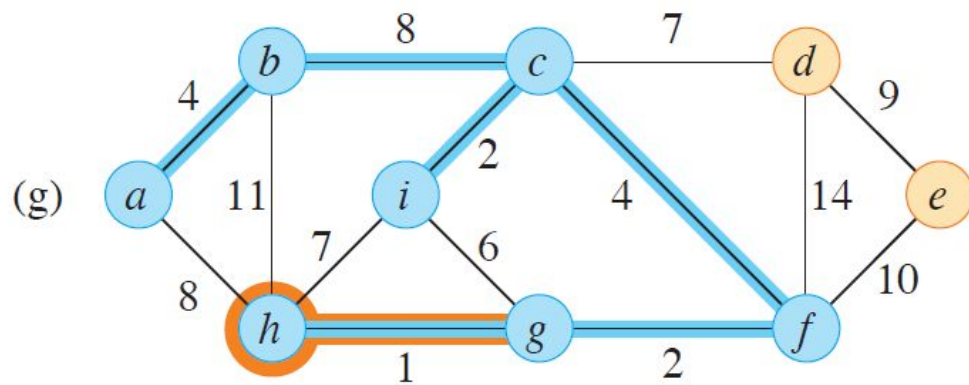
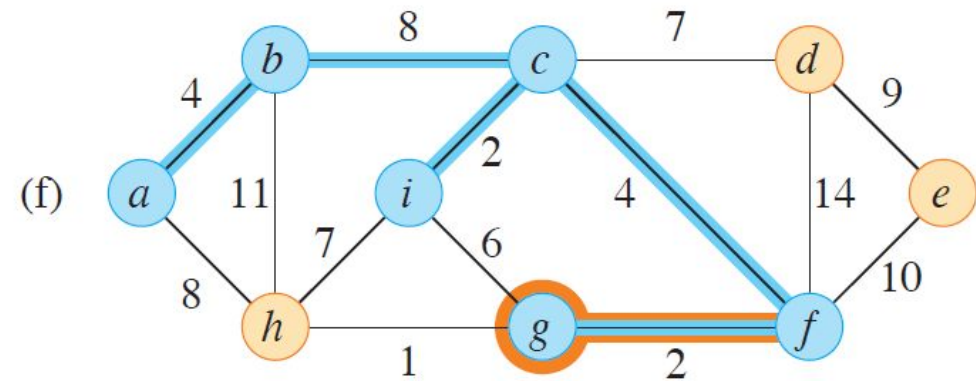
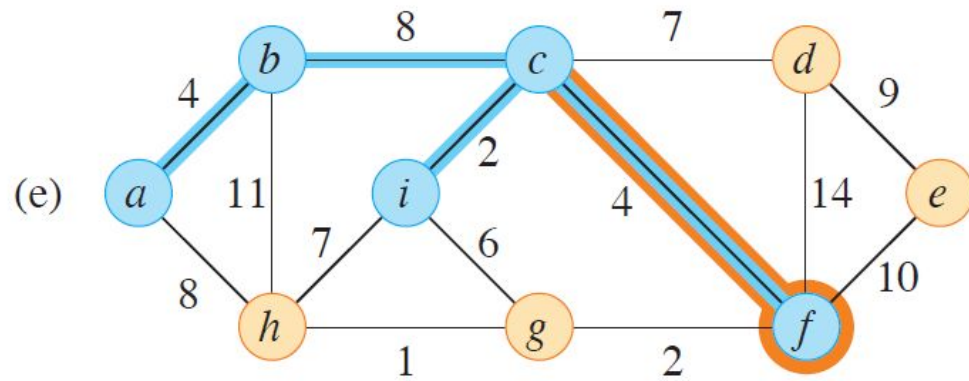
- The tree starts from an arbitrary root vertex and grows until it spans all the vertices in V
- The safe edge added to A is always a lowest-weight edge connecting the tree to a vertex not in the tree.



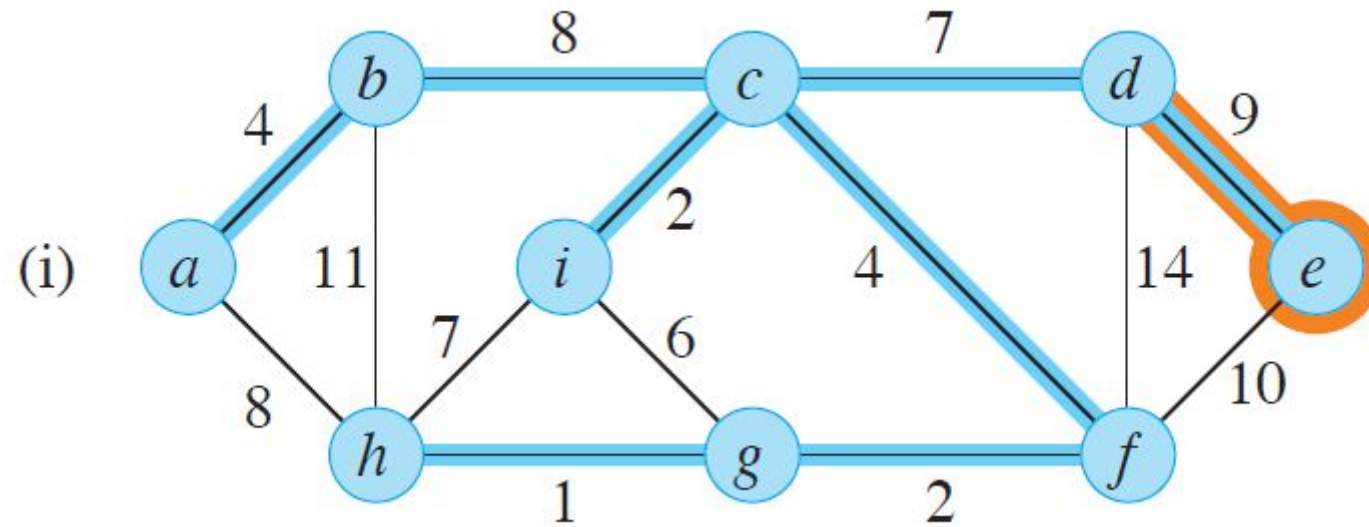
MST: Prim Algorithm: Example (1)



MST: Prim Algorithm: Example (2)



MST: Prim Algorithm: Example (3)



MST: Prim Algorithm

MST-PRIM(G, w, r)

```
1  for each vertex  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = \emptyset$ 
6  for each vertex  $u \in G.V$ 
7      INSERT( $Q, u$ )
8  while  $Q \neq \emptyset$ 
9       $u = \text{EXTRACT-MIN}(Q)$            // add  $u$  to the tree
10     for each vertex  $v$  in  $G.Adj[u]$  // update keys of  $u$ 's non-tree neighbors
11         if  $v \in Q$  and  $w(u, v) < v.key$ 
12              $v.\pi = u$ 
13              $v.key = w(u, v)$ 
14             DECREASE-KEY( $Q, v, w(u, v)$ )
```

MST: Prim Algorithm Analysis

MST-PRIM(G, w, r)

```
1  for each vertex  $u \in G.V$ 
2     $u.key = \infty$ 
3     $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = \emptyset$ 
6  for each vertex  $u \in G.V$ 
7    INSERT( $Q, u$ )
8  while  $Q \neq \emptyset$ 
9     $u = \text{EXTRACT-MIN}(Q)$ 
10   for each vertex  $v$  in  $G.Adj[u]$ 
11     if  $v \in Q$  and  $w(u, v) < v.key$ 
12        $v.\pi = u$ 
13        $v.key = w(u, v)$ 
14     DECREASE-KEY( $Q, v, w(u, v)$ )
```

- Lines 1-3 – $O(V)$ time.
- Lines 4-5 – $O(1)$ time.
- Lines 6-7 – $O(V)$ time.
- Line 8 – $O(V)$ time.
- Line 9 – $|V|$ times $O(\log V)$ time.
- Lines 10-13 – $O(E)$ times $O(1)$
- Line 14 – $O(E)$ times $O(\log V)$
- Total – $O((V + E) \log V)$
 $O(E \log V)$



Thank You